SUJET DE WORKSHOP NATIONAL · BAC+5

PROJECT RESILIENCE-ALPHA

Cyber-résilience et reprise autonome d'un système de paiement

5
domaines intégrés

13
services conteneurisés

< 60 s
reprise automatique

Équipe Workshop2026-M2-G01 · Session Octobre 2026 · Démonstrateur à données fictives

LE MESSAGE

Une plateforme qui se défend et se répare seule

Aujourd'hui, quand un service de paiement tombe, un humain doit intervenir : c'est lent, manuel et difficilement reproductible. Nous démontrons l'inverse.

1

Détecter

Un modèle de machine learning repère le comportement anormal sur les métriques temps réel.

2

Décider & agir

Un moteur de remédiation choisit l'action corrective et l'exécute sur les conteneurs.

3

Vérifier la reprise

La plateforme prouve que le métier est rétabli : santé profonde + transaction fictive réussie.

Quand un paiement ne passe plus

À l'automne 2026, une plateforme de paiement d'une société financière fictive subit des incidents répétés. Les équipes remettent le service en route, mais à la main — longuement et sans traçabilité.

- Opérations partenaires retardées
- Perte de confiance des utilisateurs
- Chaque minute d'indisponibilité coûte
- Exigences de sécurité & traçabilité (secteur financier régulé)

NOTRE MISSION

Montrer, sur un périmètre maîtrisé, qu'une reprise automatique peut être conçue, observée et vérifiée.

L'objectif n'est pas un système infaillible, mais une chaîne de cyber-résilience crédible, exécutée et défendable.

Cinq domaines, une plateforme unique

Développement

Microservices FastAPI, auth JWT, base, vitrine & dashboard React/Tailwind

Infrastructure

Docker Compose, Ansible, Prometheus / Grafana / Loki

IA / Data

Détection d'anomalies (ML) + assistant RAG (Ollama + LangChain)

Cybersécurité

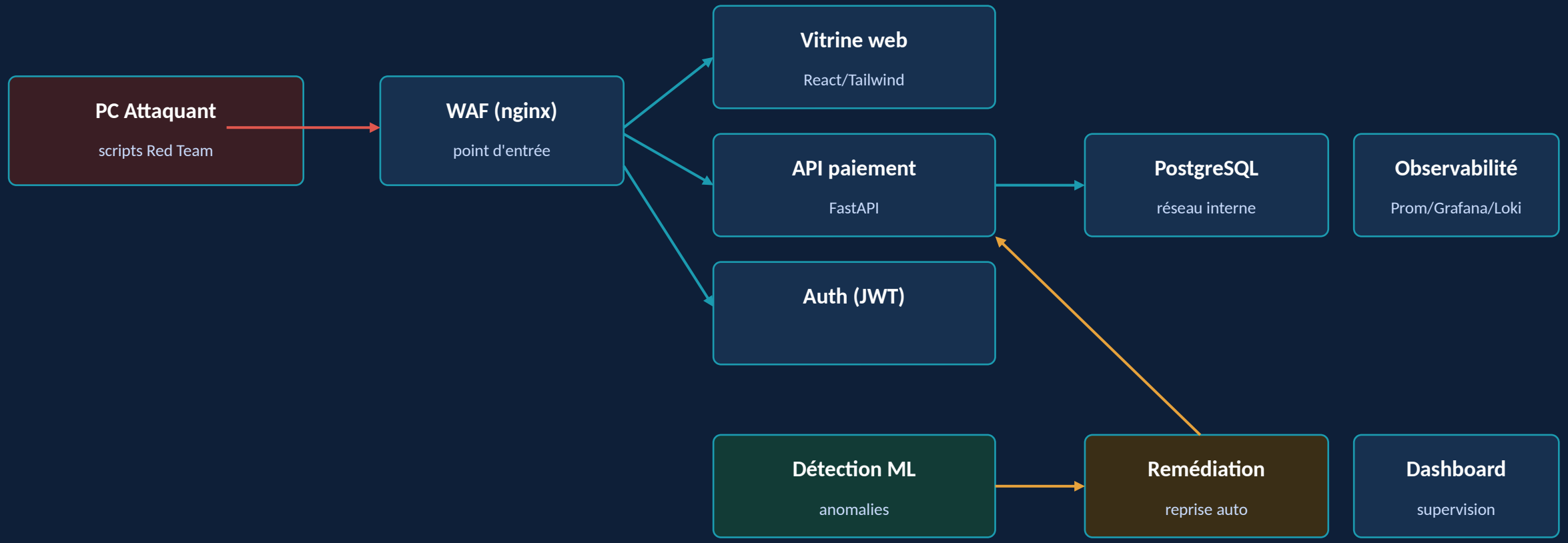
WAF, SIEM, durcissement, secrets SOPS/Vault, Red Team de labo

Gouvernance

EBIOS RM, PCA/PRA, conformité NIS2 / DORA, arbitrages

Les cinq piliers sont obligatoires, quelle que soit la composition de l'équipe.

Flux et composants



Seul le WAF est exposé · tous les autres services restent sur le réseau interne Docker (resilience_net)

Un vrai processus métier de paiement

1

API métier (FastAPI)

Création de comptes et exécution de transactions atomiques : débit/crédit sous verrou, cohérence ACID garantie par la base.

2

Authentification (JWT)

Jetons signés à courte durée ; l'API vérifie chaque transaction auprès du service auth.

3

Base PostgreSQL

Comptes et transactions. Jamais exposée hors du réseau interne.

4

Vitrine & dashboard (React/Tailwind)

Interface de paiement + supervision temps réel de toute la chaîne.

Preuve que ça marche

Transaction fictive réussie :

```
{
  "source": "acc-client-001",
  "destination": "acc-merchant-001",
  "amount": "42",
  "status": "settled"
}
```

La vitrine paie le marchand de bout en bout : vitrine → WAF → API → base.

DOMAINE · INFRASTRUCTURE

Déployer, observer, rétablir

Déploiement reproductible

- Docker Compose orchestre les 13 conteneurs
- Playbooks Ansible pour les opérations (déploiement, redémarrage, isolation)
- Tout se relance par une seule commande

Observabilité

- Prometheus collecte les métriques (débit, latence, erreurs)
- Grafana visualise + alertes (TrafficSpike, HighLatency…)
- Loki centralise les journaux via Promtail (fonction SIEM)

DOMAINE · IA / DATA

Détecter l'anomalie par apprentissage

Pas de règles codées en dur. Un modèle non supervisé (covariance robuste / EllipticEnvelope) apprend la « forme » du comportement normal et signale tout écart significatif.

`req_rate`

débit de requêtes

`error_ratio`

taux d'erreurs 5xx

`latency_p95`

latence 95e pct

`inflight`

requêtes en cours

Comment ça marche

On trace une enveloppe autour du nuage de points « normal » (sa position et sa dispersion).
Un point loin de cette zone (distance de Mahalanobis élevée) = anomalie.

Le seuil n'est pas écrit à la main : il est calibré sur les données normales (~0,5 % de faux positifs).
Pour chaque alerte : le driver principal + une explication, puis notification à la remédiation.

DOMAINE · IA / DATA

Un assistant pour l'opérateur (RAG)

Second composant IA : un assistant local qui aide l'opérateur à retrouver l'information utile dans les procédures et l'architecture — construit avec Ollama (LLM local < 8 Md de paramètres) + LangChain.



Garde-fou de sécurité : l'assistant est en lecture seule, sans aucun droit privilégié sur le système. Il ne se confond pas avec le moteur de remédiation et ne peut pas le piloter.

La reprise autonome, en boucle fermée

1
Détection
l'anomalie est signalée

→

2
Décision
plan d'action selon le driver

→

3
Action
restart / durcissement / isolation

→

4
Vérification
santé profonde + transaction test

→

5
Clôture
ou nouvelle tentative (max 3)

Reprise RÉELLE, pas cosmétique

Une reprise n'est déclarée que si la vérification métier réussit : base lisible ET nouvelle transaction fictive aboutie. Un conteneur qui redémarre ne suffit pas.

Garde-fou du composant privilégié

Le moteur a des droits élevés : il n'expose donc AUCUN shell distant, seulement une liste fermée d'actions, toutes journalisées, avec un jeton à portée limitée.

DOMAINE · CYBERSÉCURITÉ

Éprouver la plateforme sans sortir du labo

WAF applicatif

Filtrage des injections (SQLi, XSS, traversée) + limitation de débit en entrée

Durcissement

Conteneurs non-root, capacités réduites, no-new-privileges, surface minimale

SIEM / traçabilité

Journaux JSON centralisés dans Loki/Grafana, requêtes bloquées corrélables

Gestion des secrets

SOPS/Vault : aucun secret en clair dans le dépôt

Toute action offensive est strictement limitée à notre SI fictif et aux machines autorisées de l'équipe.

Deux scénarios d'attaque de laboratoire

A

Déni de service volumétrique

Rafale de requêtes via le WAF. Objectif : montrer la défense périmétrique qui absorbe le flood.

Le WAF limite le débit (429)

B

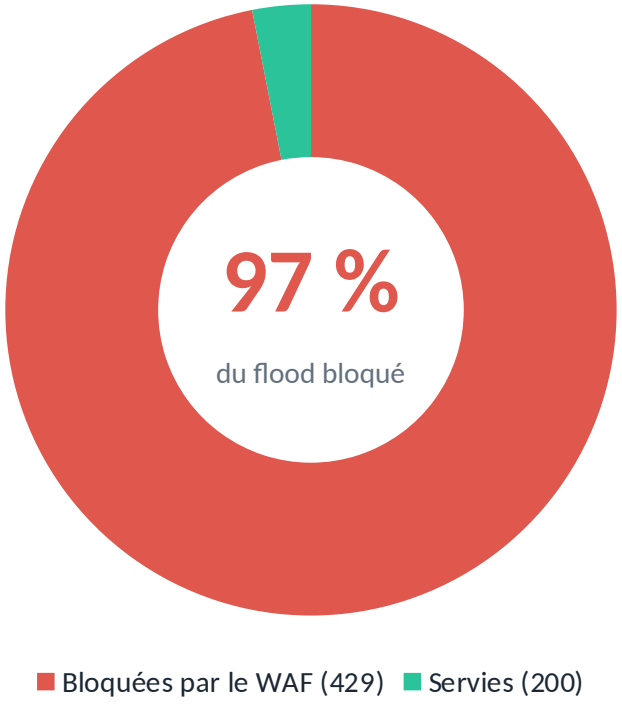
Attaque qui franchit le périmètre

Charge dirigée droit sur l'API. Objectif : déclencher la détection ML et la remédiation autonome.

Anomalie détectée → reprise auto

Ce que la démonstration prouve

Scénario A — attaque via le WAF (30 s)



< 60 s
RTO : reprise automatique sans intervention humaine

~ 0
RPO : aucune perte de données (base transactionnelle)

13 / 13
services sains après reprise, transaction fictive re-jouée

DOMAINE · GOUVERNANCE

Justifier les choix : risques & conformité

EBIOS Risk Manager

Analyse structurée : valeurs métier, sources de risque, scénarios (DoS, injection, fuite de secret, abus du moteur), traitement.

PCA / PRA

Plan adapté à la solution : RTO < 60 s, RPO ~0, scénarios couverts, et distinction disponibilité vs compromission.

NIS2 / DORA

Ce qu'un prototype de 4 jours démontre (gestion du risque, détection, tests de résilience) vs ce qu'il ne peut qu'esquisser (notification réglementaire).

Démarche de conseil : quels arbitrages, quelles recommandations à une direction, sur quels éléments vérifiables.

Limites assumées & arbitrages

Secrets en SOPS, pas Vault dynamique

Arbitrage temps/valeur pour 4 jours ; Vault recommandé en production.

Pas de mTLS interne

Réseau Docker isolé jugé suffisant dans le périmètre labo.

PC Hôte unique

Pas de redondance matérielle ; dépendance assumée.

Modèle entraîné sur observations constituées

Faute d'historique de production ; remplaçable par un export Prometheus réel.

Ce que nous revendiquons

Pas une plateforme industrielle prête à la production, mais une chaîne de cyber-résilience crédible, exécutée, testée et défendable techniquement.

Essais : détection (4 cas), validation des transactions, récupération RAG, garde-fous Red Team — tous au vert.

CE QUE LE JURY RETIENDRA

Un système de paiement critique qui encaisse un incident et se rétablit seul.

- Détection par machine learning, pas de règles codées en dur
- Remédiation autonome vérifiée par une vraie transaction métier
- Défense en profondeur : le WAF a bloqué 97 % du flood
- Gouvernance honnête : EBIOS, PCA/PRA, NIS2/DORA et limites assumées

Merci — place à la démonstration en direct.